

Drone-based Equipment Rental Database System Final Report

Instructor: Leon Madrid (madrid.1@osu.edu)

Class: OSU Autumn CSE 3241

Duration: 08/2025 - 12/2025

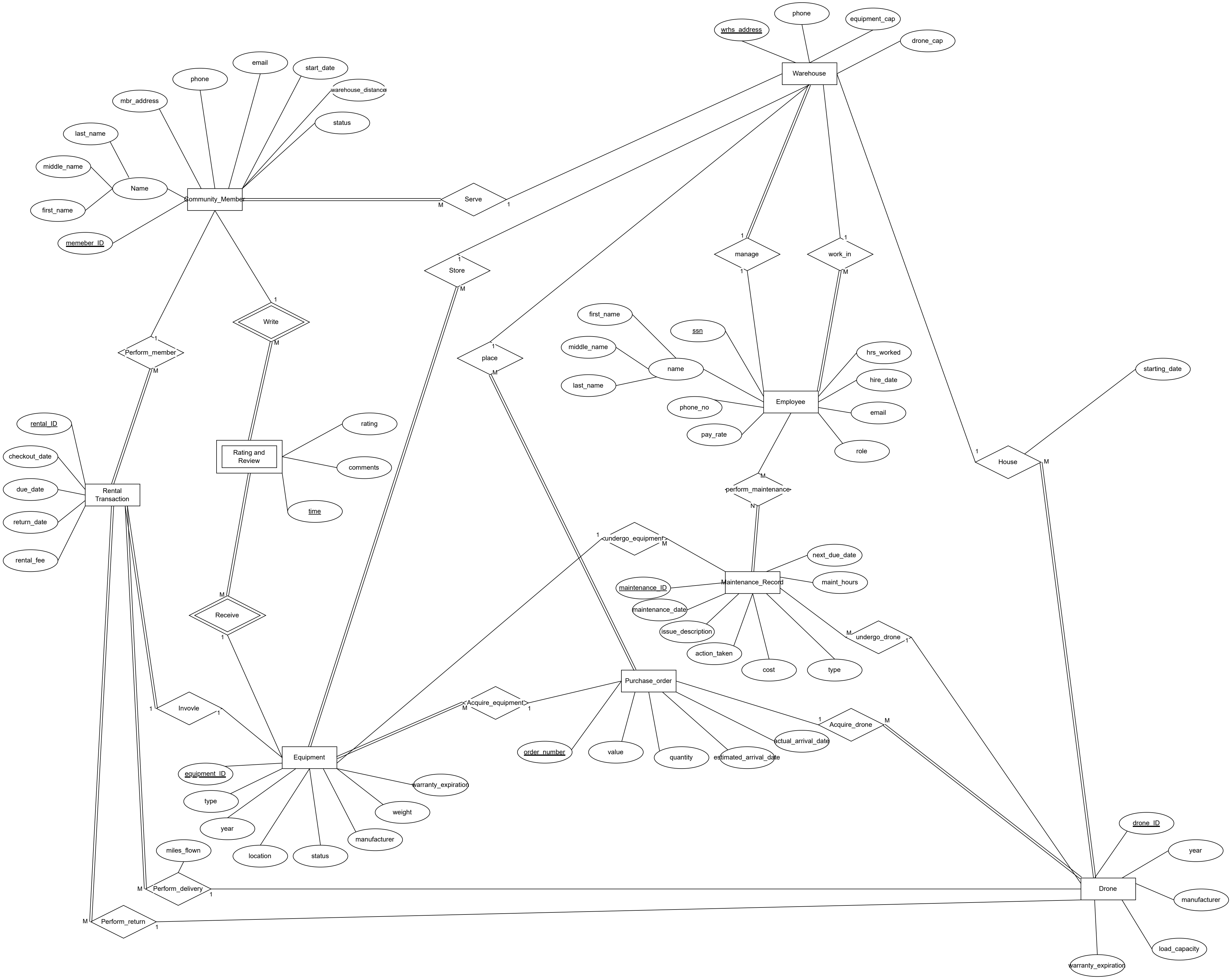
Group members:

- Qifan Wen (wen.679@osu.edu)
- Joseph Suter (suter.54@osu.edu)
- Khadija Samb (samb.3@osu.edu)
- Jeffery Guo (guo.2069@osu.edu)

GitHub: <https://github.com/Albatross679/OSU-AU25-CSE-3241-Drone-based-Equipment-Rental-Database-System->

Outline

- 1 Database Description
- 2 User Manual
- 3 Avoiding SQL Injection
- 4 Submitted Files Specifications



1 Database Description

1. ER-Model

- **Entities:**

- **Community_Member:** Attributes include `member_id` (PK), `wrhs_address` (FK), name (`f_name` , `m_name` , `l_name`), contact info (`phone` , `email`), address (`mbr_address`), `start_date` , `mbr_status` , and derived metrics like `wrhs_distance` .
- **Equipment:** Attributes include `equipment_id` (PK), `type` , `year` , `order_num` (FK), `wrhs_address` (FK), `manufacturer` , `status` , `location` , `weight` , and `warranty_expiration` .
- **Drone:** Attributes include `drone_id` (PK), `manufacturer` , `year` , `load_capacity` , and `warranty_expir` , `wrhs_address` (FK), and `order_num` (FK),
- **Warehouse:** Attributes include `wrhs_address` (PK), `phone` , `manager_ssn` (FK), and capacities (`equipment_cap` , `drone_cap`).
- **Employee:** Attributes include `ssn` (PK), name(`f_name` , `m_name` , `l_name`), `wrhs_address` (FK), `role` , `phone_no` , `email` , `hrs_worked` , `pay_rate` , and `hire_date` .
- **Rental_Transaction:** Attributes include `rental_id` (PK), `checkout_date` , `equipment_id` (FK), `member_id` (FK), `due_date` , `return_date` , and `rental_fee` .
- **Purchase_Order:** Attributes include `order_num` (PK), `quantity` , `wrhs_address` (FK), `value` , and dates (`est_arrive_date` , `actual_arrive_date`).
- **Maintenance_Record:** Attributes include `maint_id` (PK), `employee_ssn` (FK), `issue` , `action` , `next_due_date` , `cost` , and `maint_hrs` .
- **Warehouse_Houses_Drone:** Attributes include composite key (`wrhs_address` , `drone_id`) plus `start_date` . Represents where each drone is housed.
- **Drone_Performs_Delivery:** Attributes include composite key (`drone_id` , `rental_id`) and `miles_flown` . Associative entity between `Drone` and `Rental_Transaction` for deliveries.
- **Drone_Performs_Return:** Attributes include composite key (`drone_id` , `rental_id`). Associative entity between `Drone` and `Rental_Transaction` for returns.
- **Equip_Undergo_Maint:** Attributes include composite key (`maint_id` , `equipment_id`). Links `Maintenance_Record` and `Equipment` .
- **Drone_Undergo_Maint:** Attributes include composite key (`maint_id` , `drone_id`). Links `Maintenance_Record` and `Drone` .
- **Rating_And_Review:** Attributes include composite key (`member_id` , `equipment_id` , `date`), plus `rating` and `comments` .

- **Relationships:**

- **Houses:** Warehouse (1) ↔ Equipment (N).
- **Bases:** Warehouse (1) ↔ Drone (N)(Associative entity: `Warehouse_Houses_Drone` with `start_date`).
- **Employs:** Warehouse (1) ↔ Employee (N).
- **Manages:** Employee (1) ↔ Warehouse (1) (via `manager_ssn`).
- **Rents:** Community_Member (1) ↔ Rental_Transaction (N).
- **Includes:** Equipment (1) ↔ Rental_Transaction (N) (via `equipment_id`)
- **Delivers:** Drone (M) ↔ Rental_Transaction (N) (Associative entity: `Drone_Performs_Delivery` with `miles_flown`).
- **Returns:** Drone (M) ↔ Rental_Transaction (N) (Associative entity: `Drone_Performs_Return`).
- **Reviews:** Community_Member (M) ↔ Equipment (N) (Associative entity: `Rating_And_Review`).
- **Orders:** Warehouse (1) ↔ Purchase_Order (N) (via `wrhs_address`).
- **Purchased Via (Equipment):** Purchase_Order (1) ↔ Equipment (N) (via `order_num`).
- **Purchased Via (Drone):** Purchase_Order (1) ↔ Drone (N) (via `order_num`).

- **Undergoes Maintenance (Equipment):** Equipment (M) ↔ Maintenance_Record (N) (Associative entity: `Equip_Undergo_Maint`).
- **Undergoes Maintenance (Drone):** Drone (M) ↔ Maintenance_Record (N) (Associative entity: `Drone_Undergo_Maint`).

2. Relational Schema

- **Warehouse** (`wrhs_address` , `phone` , `manager_ssn` , `equipment_cap` , `drone_cap`)
 - **PK:** `wrhs_address`
 - **FK:** `manager_ssn` → `Employee(ssn)`
- **Employee** (`ssn` , `f_name` , `m_name` , `l_name` , `wrhs_address` , `role` , `phone_no` , `email` , `hire_date` , `pay_rate` , `hrs_worked`)
 - **PK:** `ssn`
 - **FK:** `wrhs_address` → `Warehouse(wrhs_address)`
- **Community_Member** (`member_id` , `wrhs_address` , `f_name` , `m_name` , `l_name` , `mbr_address` , `phone` , `email` , `start_date` , `mbr_status` , `wrhs_distance`)
 - **PK:** `member_id`
 - **FK:** `wrhs_address` → `Warehouse(wrhs_address)`
- **Purchase_Order** (`order_num` , `quantity` , `wrhs_address` , `value` , `est_arrive_date` , `actual_arrive_date`)
 - **PK:** `order_num`
 - **FK:** `wrhs_address` → `Warehouse(wrhs_address)`
- **Drone** (`drone_id` , `manufacturer` , `year` , `load_capacity` , `warranty_expir` , `wrhs_address` , `order_num`)
 - **PK:** `drone_id`
 - **FK:** `wrhs_address` → `Warehouse(wrhs_address)`
 - **FK:** `order_num` → `Purchase_Order(order_num)`
- **Equipment** (`equipment_id` , `type` , `year` , `order_num` , `wrhs_address` , `status` , `location` , `manufacturer` , `weight` , `warranty_expiration`)
 - **PK:** `equipment_id`
 - **FK:** `order_num` → `Purchase_Order(order_num)`
 - **FK:** `wrhs_address` → `Warehouse(wrhs_address)`

(Relational design treats `wrhs_address` as referencing `Warehouse`, even though the raw SQL has a typo in the FK.)
- **Rental_Transaction** (`rental_id` , `checkout_date` , `equipment_id` , `member_id` , `due_date` , `return_date` , `rental_fee`)
 - **PK:** `rental_id`
 - **FK:** `equipment_id` → `Equipment(equipment_id)`
 - **FK:** `member_id` → `Community_Member(member_id)`
- **Drone_Performs_Delivery** (`rental_id` , `drone_id` , `miles_flown`)
 - **PK:** (`drone_id` , `rental_id`)
 - **FK:** `rental_id` → `Rental_Transaction(rental_id)`
 - **FK:** `drone_id` → `Drone(drone_id)`
- **Drone_Performs_Return** (`rental_id` , `drone_id`)
 - **PK:** (`drone_id` , `rental_id`)
 - **FK:** `rental_id` → `Rental_Transaction(rental_id)`
 - **FK:** `drone_id` → `Drone(drone_id)`

- **Warehouse_Houses_Drone** (wrhs_address , drone_id , start_date)
 - **PK:** (wrhs_address , drone_id)
 - **FK:** wrhs_address → Warehouse(wrhs_address)
 - **FK:** drone_id → Drone(drone_id)
- **Maintenance_Record** (maint_id , employee_ssn , maint_date , issue , action , next_due_date , cost , maint_hrs)
 - **PK:** maint_id
 - **FK:** employee_ssn → Employee(ssn)

(Again, ER/relational design uses the corrected name employee_ssn .)
- **Equip_Undergo_Maint** (maint_id , equipment_id)
 - **PK:** (maint_id , equipment_id)
 - **FK:** maint_id → Maintenance_Record(maint_id)
 - **FK:** equipment_id → Equipment(equipment_id)
- **Drone_Undergo_Maint** (maint_id , drone_id)
 - **PK:** (maint_id , drone_id)
 - **FK:** maint_id → Maintenance_Record(maint_id)
 - **FK:** drone_id → Drone(drone_id)
- **Rating_And_Review** (member_id , equipment_id , date , rating , comments)
 - **PK:** (member_id , equipment_id , date)
 - **FK:** member_id → Community_Member(member_id)
 - **FK:** equipment_id → Equipment(equipment_id)

3. Normalization

The schema has been designed to achieve **Boyce-Codd Normal Form (BCNF)** to minimize redundancy and update anomalies.

- **Table: Rental_Transaction**
 - **Functional Dependencies:** rental_id → {checkout_date, equipment_id, member_id, due_date, rental_fee}
 - **Justification:** The table is in BCNF. The primary key rental_id determines all other non-prime attributes. There are no transitive dependencies (e.g., member_id does not determine equipment_id).
- **Table: Drone**
 - **Functional Dependencies:** drone_id → {manufacturer, year, load_capacity, wrhs_address, order_num}
 - **Justification:** The table is in BCNF. While wrhs_address implies a location, it is determined by the specific assignment of the drone (the drone_id), not by the manufacturer or order number.
- **Table: Purchase_Order**
 - **Functional Dependencies:** order_num → {quantity, wrhs_address, value, dates...}
 - **Justification:** The table is in BCNF. The order number serves as the unique identifier for the transaction. Note that wrhs_address is here because an order is destined for a specific location; knowing the warehouse does not determine the unique order number or value.

4. Indexes

Three specific indexes (excluding Primary Keys) have been designed to optimize the performance of frequent queries identified in [Queries and Interesting Queries.sql](#) .

1. Index on `Equipment(type)`

- **Rationale:** Query #7 results by `Equipment.type` and Query #1 filters by `Manufacturer` (e.g., `WHERE manufacturer = 'Ridgid'`). Although Query #1 does not filter on `Type`, `Type` is still an important categorical search attribute. Indexing equipment type helps reducing number of table scans for type searches and grouping operations.

```
CREATE INDEX idx_equipment_type
ON Equipment(type);
```

2. Index on `Rental_Transaction(member_id)`

- **Rationale:** Query #2, #4, and #6 involve filtering or joining based on `member_id` (e.g., `WHERE RT.member_id = 923927`). As the transaction history grows, finding a specific member's history requires high-performance lookups. This index supports the "Member History" feature essential for the application.

```
CREATE INDEX idx_rental_member
ON Rental_Transaction(member_id);
```

3. Index on `Drone(manufacturer)`

- **Rationale:** "Interesting Query 1" calculates averages grouped by `manufacturer`. Additionally, inventory analysis often requires filtering by brand for warranty or maintenance recall purposes. This index optimizes aggregation queries related to fleet management.

```
CREATE INDEX idx_drone_manufacturer
ON Drone(manufacturer);
```

5. Views

Two views were explicitly implemented in the final SQL dump to simplify complex reporting for management.

i. View: `Manager_Info`

- **Description:** This view joins the Warehouse and Employee tables to provide a consolidated report of warehouse locations, the name of the manager assigned to that warehouse, and the average salary cost for that location.
- **Relational Algebra:**

$$\pi_{wrhs_address, Manager_name, Avg_Salary} (\gamma_{wrhs_address, Manager_name; AVG(pay_rate \times hrs_worked) \rightarrow Avg_Salary} (Warehouse \bowtie Manager_ssn=$$

- **SQL Statement:**

```
CREATE VIEW Manager_Info AS
SELECT warehouse.Wrhs_address,
       CONCAT(Employee.F_name, ' ', Employee.L_name) AS Manager_name,
       AVG(Employee.Pay_rate * Employee.Hrs_worked) AS Avg_Salary
FROM Warehouse, Employee
WHERE Warehouse.Wrhs_address = Employee.Wrhs_address
  AND Warehouse.Manager_ssn = Employee.Ssn
GROUP BY Warehouse.wrhs_address, Manager_name;
```

- **Sample Output:**

Wrhs_address	Manager_name	Avg_Salary
1924 High St...	Avery Johnson	878.80
9774 Spring St...	Rowan Rodriguez	1919.25

ii. View: Warranty_Expring_2026

- **Description:** This view filters the drone fleet to identify units with warranties expiring in the year 2026, grouped by their home warehouse. It allows logistics coordinators to anticipate replacement costs.
- **Relational Algebra:**

$\Pi_{\text{wrhs_address}, \text{Num_of_Drones}} \left(\gamma_{\text{wrhs_address}; \text{COUNT}(\text{drone_id}) \rightarrow \text{Num_of_Drones}} \left(\text{Warehouse} \bowtie \left(\sigma_{\text{warranty_expir LIKE '2026-\%'}}(\text{Drone}) \right) \right) \right)$

- **SQL Statement:**

```
CREATE VIEW Warranty_Expring_2026 AS
SELECT warehouse.wrhs_address,
       COUNT(drone.drone_id) AS Num_of_Drones
FROM warehouse, drone
WHERE warehouse.wrhs_address = drone.wrhs_address
AND Drone.warranty_expir LIKE '2026-%'
GROUP BY Warehouse.wrhs_address;
```

- **Sample Output:**

wrhs_address	Num_of_Drones
1684 Main St...	5
1391 3rd St...	3

6. Transactions

The following transactions illustrate units of work that require ACID properties to ensure database consistency.

Transaction 1: Processing a New Equipment Rental

- **Description:** This transaction handles the checkout process. It verifies the equipment is available (not shown in SQL but part of logic), inserts a new record into `Rental_Transaction`, and immediately updates the `Equipment` status to 'in_use' to prevent double-booking. (Note: If the transaction failed, the client should issue `ROLLBACK` instead of `COMMIT`, to avoid leaving the database in a partial condition)
- **SQL Code:**

```
BEGIN TRANSACTION;
-- Step 1: Create the rental record
INSERT INTO Rental_Transaction (rental_id, checkout_date, equipment_id, member_id, due_date, rental_fee)
VALUES (100200, DATE('now'), 5022, 923927, DATE('now', '+7 days'), 45.00);

-- Step 2: Mark equipment as unavailable
UPDATE Equipment
SET status = 'in_use'
WHERE equipment_id = 5022;
COMMIT;
```

Transaction 2: Processing an Equipment Return via Drone

- **Description:** When a drone returns an item, three things must happen atomically: the rental transaction is closed with a return date, the equipment status is reset to 'available', and the drone's flight log is updated to record the return trip.
- **SQL Code:**


```

BEGIN TRANSACTION;
-- Step 1: Close the rental
UPDATE Rental_Transaction
SET return_date = DATE('now')
WHERE rental_id = 100200;

-- Step 2: Update equipment status
UPDATE Equipment
SET status = 'available', location = '1684 Main St, Gahanna, OH 43001 #5'
WHERE equipment_id = 5022;

-- Step 3: Log the drone flight
INSERT INTO Drone_Performs_Return (rental_id, drone_id)
VALUES (100200, 1011);
COMMIT;

```

Transaction 3: Scheduling Equipment Maintenance

- **Description:** When equipment is flagged for repair, a maintenance record is created, and the equipment is simultaneously removed from the rental pool by changing its status to 'maintenance'.
- **SQL Code:**

```

BEGIN TRANSACTION;
-- Step 1: Log the maintenance issue
INSERT INTO Maintenance_Record (maint_id, employee_ssn, maint_date, issue, action, cost)
VALUES (50, '755984598', DATE('now'), 'Calibration error', 'Pending Inspection', 0.00);

-- Step 2: Link equipment to maintenance record
INSERT INTO Equip_Undergo_Maint (maint_id, equipment_id)
VALUES (50, 5031);

-- Step 3: Lock equipment from future rentals
UPDATE Equipment
SET status = 'maintenance'
WHERE equipment_id = 5031;
COMMIT;

```

2 User Manual

2.1 Table Descriptions

This database models a drone delivery service inventory and rental system. Below are the details for the core entities and their attributes.

Warehouse

Represents physical storage locations for drones and equipment.

Attributes:

- `wrhs_address` (VARCHAR 100, PK): The physical street address of the facility.
- `phone` (VARCHAR 20): Contact number for the warehouse.
- `manager_ssn` (VARCHAR 9, FK): Links to the `Employee` table identifying the facility manager.
- `equipment_cap` (INT): Maximum number of equipment items stored.
- `drone_cap` (INT): Maximum number of drones stored.

Community_Member

Represents the customers or members who rent equipment.

Attributes:

- `member_id` (INT, PK): Unique identifier for the member.
- `wrhs_address` (VARCHAR 100, FK): The primary warehouse assigned to this member.
- `f_name`, `m_name`, `l_name` (VARCHAR): Member's full name.
- `mbr_address` (VARCHAR 255): The member's home address.
- `phone`, `email` (VARCHAR): Contact information.
- `start_date` (DATE): Membership initiation date.
- `mbr_status` (VARCHAR 50): Current standing (e.g., 'Active', 'Suspended').
- `wrhs_distance` (INTEGER): Distance from their assigned warehouse.

Employee

Represents staff working at the warehouses.

Attributes:

- `ssn` (VARCHAR 9, PK): Social Security Number (Unique ID).
- `f_name`, `m_name`, `l_name` (VARCHAR): Employee name.
- `wrhs_address` (VARCHAR 100, FK): The warehouse where the employee works.
- `role` (VARCHAR 50): Job title (e.g., 'Manager', 'Dispatcher').
- `pay_rate` (DECIMAL): Hourly wage.
- `hire_date` (DATE): Date of employment.

Purchase_Order

Represents external orders to acquire new inventory.

Attributes:

- `order_num` (INT, PK): Unique tracking number.

- `quantity` (INT): Number of items ordered.
- `value` (DECIMAL): Total monetary value of the order.
- `est_arrive_date` , `actual_arrive_date` (DATE): Logistics tracking dates.

Drone

Represents the autonomous vehicles used for delivery.

Attributes:

- `drone_id` (INT, PK): Unique identifier for the drone.
- `manufacturer` (VARCHAR): Maker of the drone (e.g., 'DJI', 'Autel').
- `load_capacity` (DECIMAL): Max weight the drone can carry.
- `warranty_expir` (DATE): Warranty expiration date.
- `wrhs_address` (FK): Current storage location.
- `order_num` (FK): The purchase order under which this drone was bought.

Equipment

Represents the tools and items available for rent.

Attributes:

- `equipment_id` (INT, PK): Unique identifier.
- `type` (VARCHAR): Category of tool (e.g., 'Drill', 'Ladder').
- `status` (VARCHAR): Availability (e.g., 'available', 'in_use', 'maintenance').
- `manufacturer` (VARCHAR): Brand name.
- `weight` (DECIMAL): Physical weight (must fit drone capacity).
- `order_num` (FK): Originating purchase order.

Rental_Transaction

Represents the event of a member renting a specific item.

Attributes:

- `rental_id` (INT, PK): Unique transaction ID.
- `member_id` (FK): The renting member.
- `equipment_id` (FK): The item being rented.
- `checkout_date` (DATETIME): When the rental began.
- `due_date` , `return_date` (DATE): Expected and actual return dates.
- `rental_fee` (DECIMAL): Cost of the rental.

Associative Tables

- **Drone_Performs_Delivery:** Links `Rental_Transaction` and `Drone` , recording `miles_flown` for a specific delivery.
- **Drone_Performs_Return:** Links `Rental_Transaction` and `Drone` for the return leg of the trip.
- **Maintenance_Record / Equip_Undergo_Maint:** Tracks repairs performed on equipment/drones by employees.

2.2 Sample SQL Queries

Note: See `AllQueries.txt` for the raw text of these queries.

[AllQueries.txt](#)

Basic Queries

Query 1: Filtered Equipment Search

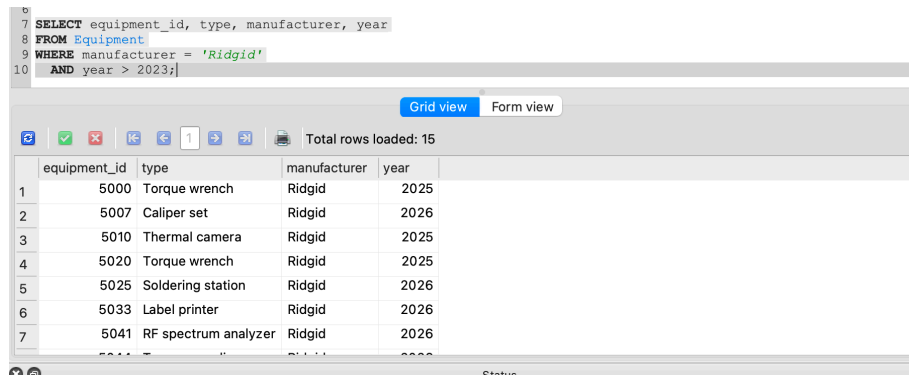
- i. **Description:** Selects the ID, type, manufacturer, and year of all equipment made by 'Ridgid' manufactured after 2023.
- ii. **Relational Algebra:**

$$\pi_{equipment_id, type, manufacturer, year}(\sigma_{manufacturer='Ridgid' \wedge year > 2023}(Equipment))$$

- iii. **SQL Query:**

```
SELECT equipment_id, type, manufacturer, year
FROM Equipment
WHERE manufacturer = 'Ridgid'
AND year > 2023;
```

- iv. **Status:** Tested and correct.
- v. **Screenshot:**



The screenshot shows a database query interface. At the top, the SQL query is displayed: `SELECT equipment_id, type, manufacturer, year FROM Equipment WHERE manufacturer = 'Ridgid' AND year > 2023;`. Below the query, there are buttons for 'Grid view' and 'Form view'. The 'Grid view' button is selected. Below the buttons, a status bar indicates 'Total rows loaded: 15'. The main area displays a table with 5 columns: 'equipment_id', 'type', 'manufacturer', 'year', and an empty column. The table contains 7 rows of data, all with 'Ridgid' as the manufacturer and years greater than 2023.

	equipment_id	type	manufacturer	year	
1	5000	Torque wrench	Ridgid	2025	
2	5007	Caliper set	Ridgid	2026	
3	5010	Thermal camera	Ridgid	2025	
4	5020	Torque wrench	Ridgid	2025	
5	5025	Soldering station	Ridgid	2026	
6	5033	Label printer	Ridgid	2026	
7	5041	RF spectrum analyzer	Ridgid	2026	

Query 2: Rental Join

- i. **Description:** Retrieves the equipment ID, type, and checkout date for all rentals made by the member with ID 923927.
- ii. **Relational Algebra:**

$$\pi_{RT.equipment_id, E.type, RT.checkout_date}(\sigma_{RT.member_id=923927}(Rental_Transaction \bowtie_{RT.equipment_id=E.equipment_id} Equipment))$$

- iii. **SQL Query:**

```
SELECT RT.equipment_id, E.type, RT.checkout_date
FROM Rental_Transaction AS RT
JOIN Equipment AS E ON RT.equipment_id = E.equipment_id
WHERE RT.member_id = 923927;
```

- iv. **Status:** Tested and correct.
- v. **Screenshot:**

```

11
12 SELECT RT.equipment_id, E.type, RT.checkout_date
13 FROM Rental_Transaction AS RT
14 JOIN Equipment AS E ON RT.equipment_id = E.equipment_id
15 WHERE RT.member_id = 923927;

```

Grid view Form view

Total rows loaded: 1

equipment_id	type	checkout_date
1	5076 Power drill	2025-06-23 12:01:39

Query 3: Drones in Low-Traffic Warehouses

- i. **Description:** Lists drones stored in warehouses that house 2 or fewer drones, showing warehouse address, drone ID, and load capacity.
- ii. **Relational Algebra:**

$$R_{low_cap} \leftarrow \pi_{wrhs_address}(\sigma_{count \leq 2}(wrhs_address \bowtie_{COUNT(drone_id) \rightarrow count} (Warehouse_Houses_Drone)))$$

$$\pi_{WHD.wrhs_address, D.drone_ID, D.load_capacity} (Drone \bowtie_{D.drone_id = WHD.drone_id} (Warehouse_Houses_Drone \bowtie R_{low_ca}))$$

- iii. **SQL Query:**

```

SELECT WHD.wrhs_address, D.drone_ID, D.load_capacity
FROM Drone AS D
JOIN Warehouse_Houses_Drone AS WHD ON D.drone_id = WHD.drone_id
WHERE WHD.wrhs_address IN (
    SELECT wrhs_address
    FROM Warehouse_Houses_Drone
    GROUP BY wrhs_address
    HAVING COUNT(drone_id) <= 2
)
ORDER BY WHD.wrhs_address, D.drone_id;

```

- iv. **Status:** Tested and correct.
- v. **Screenshot:**

```

6
7 SELECT WHD.wrhs_address, D.drone_ID, D.load_capacity
8 FROM Drone AS D
9 JOIN Warehouse_Houses_Drone AS WHD ON D.drone_id = WHD.drone_id
10 WHERE WHD.wrhs_address IN (
11     SELECT wrhs_address
12     FROM Warehouse_Houses_Drone
13     GROUP BY wrhs_address
14     HAVING COUNT(drone_id) <= 2
15 )
16 ORDER BY WHD.wrhs_address, D.drone_id;

```

Grid view Form view

Total rows loaded: 2

wrhs_address	drone_id	load_capacity
3833 Olentangy River Rd, Reynoldsburg, OH 43004 #7	1034	18.34
3833 Olentangy River Rd, Reynoldsburg, OH 43004 #7	1037	10.14

Query 4: Long Distance Deliveries

- **i. Description:** Finds member name, equipment type, miles flown, and checkout date for rentals where the member lives more than 15 miles from the warehouse and the delivery was performed by drone 1012.
- **ii. Relational Algebra:**

$$\pi_{RT.member_id, M.f_name, M.l_name, RT.equipment_id, E.type, DD.miles_flown, RT.checkout_date}(\sigma_{M.wrhs_distance > 15 \wedge DD.drone_id = 1012}(1$$

- **iii. SQL Query:**

```
SELECT RT.member_id, M.f_name, M.l_name, RT.equipment_id, E.type, DD.miles_flown, RT.checkout_date
FROM Rental_Transaction AS RT
JOIN Community_Member AS M ON RT.member_id = M.member_id
JOIN Equipment AS E ON RT.equipment_id = E.equipment_id
JOIN Drone_Performs_Delivery AS DD ON RT.rental_id = DD.rental_id
WHERE M.wrhs_distance > 15
AND DD.drone_id = 1012;
```

- **iv. Status:** Tested and correct.
- **v. Screenshot:**

The screenshot shows a database query interface. At the top, the SQL query is entered in a text area. Below the query, there are buttons for 'Grid view' and 'Form view'. The 'Grid view' button is active. Below the buttons, a status bar indicates 'Total rows loaded: 2'. Below the status bar, a table displays the results of the query. The table has 7 columns: member_id, f_name, l_name, equipment_id, type, miles_flown, and checkout_date. There are 2 rows of data.

	member_id	f_name	l_name	equipment_id	type	miles_flown	checkout_date
1	284229	Jamie	Wilson	5080	LED work light	27.92	2024-03-03 16:50:01
2	800005	Jamie	Anderson	5082	Soldering station	43.11	2024-02-22 18:29:51

Query 5: Unique Drones for Specific Address

- **i. Description:** Counts the number of unique drones that performed returns for members living at addresses containing "1757 3rd St".
- **ii. Relational Algebra:**

$$\mathcal{S}_{COUNT(drone_id)}(\pi_{DR.drone_id}(Drone_Performs_Return \bowtie Rental_Transaction \bowtie \sigma_{mbr_address LIKE '%17573rdSt\%'}$$

- **iii. SQL Query:**

```
SELECT COUNT(DISTINCT DR.drone_id) AS Unique_Drones
FROM Drone_Performs_Return AS DR
JOIN Rental_Transaction AS RT ON DR.rental_id = RT.rental_id
JOIN Community_Member AS M ON RT.member_id = M.member_id
WHERE M.mbr_address LIKE '%1757 3rd St%';
```

- **iv. Status:** Tested and correct.
- **v. Screenshot:**

```

36 SELECT COUNT(DISTINCT DR.drone_id) AS Unique_Drones
37 FROM Drone_Performs_Return AS DR
38 JOIN Rental_Transaction AS RT ON DR.rental_id = RT.rental_id
39 JOIN Community_Member AS M ON RT.member_id = M.member_id
40 WHERE M.mbr_address LIKE '%1757 3rd St%';

```

Unique_Drones
1

Total rows loaded: 1

Status

Query 6: Member with Most Rentals

- i. **Description:** Identifies the member who has rented the most items, showing their ID, first name, last name, and total rental count.
- ii. **Relational Algebra:**

$$\pi_{M.member_id, M.f_name, M.l_name, Rentals}(Community_Member \bowtie (member_id \xrightarrow{COUNT(rental_id)} Rentals(Rental_Tran.$$

- iii. **SQL Query:**

```

SELECT M.member_id, M.f_name, M.l_name, COUNT(R.rental_id) AS Rentals
FROM Community_Member AS M
JOIN Rental_Transaction AS R ON R.member_id = M.member_id
GROUP BY R.member_id
ORDER BY COUNT(R.rental_id) DESC
LIMIT 1;

```

- iv. **Status:** Tested and correct.
- v. **Screenshot:**

```

2 SELECT M.member_id, M.f_name, M.l_name, COUNT(R.rental_id) AS Rentals
3 FROM Community_Member AS M
4 JOIN Rental_Transaction AS R ON R.member_id = M.member_id
5 GROUP BY R.member_id
6 ORDER BY COUNT(R.rental_id) DESC
7 LIMIT 1;

```

member_id	f_name	l_name	Rentals
800005	Jamie	Anderson	6

Total rows loaded: 1

Status

Query 7: Recent Equipment Statistics by Type

- i. **Description:** Shows equipment type, total quantity ordered, total value, and average weight for equipment that arrived within the last 6 months.
- ii. **Relational Algebra:**

$$\pi_{type, Total_Quantity, Total_Value, Average_Weight}(E.type \xrightarrow{SUM(P.quantity), SUM(P.value), AVG(E.weight)} (Equipment \bowtie_{E.order_num=}$$

- iii. **SQL Query:**

```

SELECT E.type,
       SUM(P.quantity) AS Total_Quantity,
       SUM(P.value) AS Total_Value,
       AVG(E.weight) AS Average_Weight
FROM Equipment AS E
JOIN Purchase_Order AS P ON E.order_num = P.order_num
WHERE P.actual_arrive_date > DATE('now', '-6 months') -- or a fixed date like '2024-05-05'
GROUP BY E.type;

```

- **iv. Status:** Tested and correct.
- **v. Screenshot:**

The screenshot shows a SQL query execution interface. The query is as follows:

```

SELECT E.type,
       SUM(P.quantity) AS Total_Quantity,
       SUM(P.value) AS Total_Value,
       AVG(E.weight) AS Average_Weight
FROM Equipment AS E
JOIN Purchase_Order AS P ON E.order_num = P.order_num
WHERE P.actual_arrive_date > DATE('now', '-6 months') -- or a fixed date like '2024-05-05'
GROUP BY E.type;

```

The results are displayed in a table with 4 columns: type, Total_Quantity, Total_Value, and Average_Weight. There are 6 rows of data.

	type	Total_Quantity	Total_Value	Average_Weight
1	Air compressor	86	46229.29	22.9875
2	Barcode scanner	89	29241.98	10.156666666666667
3	Battery charger	38	30238	7.24
4	Caliper set	41	29659.89	18.106666666666667
5	Fiber optic tester	25	6358.62	16
6	First aid kit	18	18876.74	9.02

Interesting Queries

Interesting Query 1: Average Drone Capacity by Manufacturer

- **i. Description:** Calculates the average load capacity of drones, grouped by manufacturer.
- **ii. Relational Algebra:**

$$manufacturer \bowtie_{AVG(load_capacity) \rightarrow Avg_Load_Capacity} (Drone)$$

- **iii. SQL Query:**

```

SELECT manufacturer, AVG(load_capacity) AS Avg_Load_Capacity
FROM Drone
GROUP BY manufacturer;

```

- **iv. Status:** Tested and correct.
- **v. Screenshot:**

The screenshot shows a SQL query execution interface. The query is as follows:

```

SELECT manufacturer, AVG(load_capacity) AS Avg_Load_Capacity
FROM Drone
GROUP BY manufacturer;

```

The results are displayed in a table with 2 columns: manufacturer and Avg_Load_Capacity. There are 6 rows of data.

	manufacturer	Avg_Load_Capacity
1	Autel Robotics	12.691666666666667
2	DJI	13.13
3	Holy Stone	13.68142857142857
4	Hubsan	6.54
5	Parrot	15.0575
6	Skydio	11.405

Interesting Query 2: Currently Unreturned Rentals

- **i. Description:** Lists rental ID, equipment type, checkout date, and member name for all rentals that have not yet been returned.
- **ii. Relational Algebra:**

$$\pi_{RT.rental_ID, E.type, RT.checkout_date, M.f_name, M.l_name}(\sigma_{RT.return_date \text{ IS NULL}}(Rental_Transaction \bowtie Equipment \bowtie ($$

- **iii. SQL Query:**

```
SELECT RT.rental_ID, E.type, RT.checkout_date, M.f_name, M.l_name
FROM Rental_Transaction AS RT
JOIN Equipment AS E ON RT.equipment_id = E.equipment_id
JOIN Community_Member AS M ON RT.member_id = M.member_id
WHERE RT.return_date IS NULL;
```

- **iv. Status:** Tested and correct.
- **v. Screenshot:**

	rental_id	type	checkout_date	f_name	l_name
1	100000	LED work light	2024-09-12 20:16:26	Henry	Moore
2	100002	Portable inverter	2025-09-05 13:32:08	Jamie	Anderson
3	100008	Barcode scanner	2025-03-10 14:46:51	Morgan	Wilson
4	100010	LED work light	2025-06-09 16:16:25	Amelia	Martin
5	100011	Caliper set	2025-06-26 14:52:27	Avery	White
6	100015	Thermal camera	2024-09-22 11:41:39	Mason	Hernandez

Interesting Query 3: Total Equipment Value by Warehouse

- **i. Description:** Calculates the total value of all equipment stored at each warehouse, based on purchase order values.
- **ii. Relational Algebra:**

$$W.wrhs_address \xrightarrow{\sum(PO.value) \rightarrow Total_Value_of_Equipment} (Warehouse \bowtie_{W.wrhs_address=E.wrhs_address} Equipment \bowtie_{E.order_num=PO.order_num} Purchase_Order)$$

- **iii. SQL Query:**

```
SELECT W.wrhs_address, SUM(PO.value) AS Total_Value_of_Equipment
FROM Warehouse AS W
JOIN Equipment AS E ON W.wrhs_address = E.wrhs_address
JOIN Purchase_Order AS PO ON E.order_num = PO.order_num
GROUP BY W.wrhs_address;
```

- **iv. Status:** Tested and correct.
- **v. Screenshot:**

```

9 SELECT W.wrhs_address, SUM(PO.value) AS Total_Value_of_Equipment
10 FROM Warehouse AS W
11 JOIN Equipment AS E ON W.wrhs_address = E.wrhs_address
12 JOIN Purchase_Order AS PO ON E.order_num = PO.order_num
13 GROUP BY W.wrhs_address;

```

	wrhs_address	Total_Value_of_Equipment
1	1391 3rd St,Reynoldsburg,OH 43215 #6	119784.16
2	1684 Main St,Gahanna,OH 43001 #5	133707.77
3	1924 High St,Reynoldsburg,OH 43004 #1	79122.66
4	3357 Indianola Ave,Westerville,OH 43210 #3	116406.78
5	3833 Olentangy River Rd,Reynoldsburg,OH 43004 #7	142644.2
6	6077 Neil Ave,Gahanna,OH 43004 #8	102753.94

Advanced Queries

Query J: Total Amount Rented per Member

- i. **Description:** Lists member ID, member name, and the sum of rental fees for all items they have rented.
- ii. **Relational Algebra:**

$$\pi_{CM.member_ID,member_name,total_amount_rented} (CM.member_ID,CM.F_name,CM.L_name \bowtie \Sigma_{SUM(RT.Rental_fee) \rightarrow total_amount_rented}$$

- iii. **SQL Query:**

```

SELECT CM.member_ID, CM.F_name || ' ' || CM.L_name AS member_name, SUM(RT.Rental_fee) AS total_amount_r
ented
FROM Community_Member AS CM
JOIN Rental_Transaction AS RT ON CM.member_ID = RT.member_ID
GROUP BY CM.member_ID, member_name;

```

- iv. **Status:** Tested and correct.
- v. **Screenshot:**

```

4
5 SELECT CM.member_ID, CM.F_name || ' ' || CM.L_name AS member_name, SUM(RT.Rental_fee) AS total_amount_rented
6 FROM Community_Member AS CM
7 JOIN Rental_Transaction AS RT ON CM.member_ID = RT.member_ID
8 GROUP BY CM.member_ID, member_name;

```

	member_id	member_name	total_amount_rented
1	112426	Jordan Wilson	109
2	124302	Amelia Thomas	73
3	132359	Isabella Jackson	262
4	156304	Riley Lopez	282
5	162535	Elliot Williams	58
6	166645	Emma White	92

Status

[09:09:43] Query finished in 0.004 second(s).

Query K: Members with Above Average Rentals

- i. **Description:** Lists member ID, name, email, and number of rentals for members who have rented more items than the average number of items rented per member.
- ii. **Relational Algebra:**

$$R_{counts} \leftarrow member_ID \bowtie \Sigma_{COUNT(*) \rightarrow num_rented} (Rental_Transaction)$$

$$V_{avg} \leftarrow \Sigma_{AVG(num_rented) \rightarrow avg_rented} (R_{counts})$$

$$\pi_{CM.member_ID,member_name,CM.Email,R.num_rented} ((R_{counts} \times V_{avg}) \bowtie_{num_rented > avg_rented} Community_Member)$$

- **iii. SQL Query:**

```
WITH rental_counts AS (
  SELECT member_ID, COUNT(*) AS num_rented
  FROM Rental_Transaction
  GROUP BY member_ID),avg_rent AS (
  SELECT AVG(num_rented) AS avg_rented FROM rental_counts
)
SELECT CM.member_ID, CM.F_name || ' ' || CM.L_name AS member_name, CM.Email,R.num_rented
FROM rental_counts AS R
JOIN avg_rent AS A
JOIN Community_Member AS CM ON CM.member_ID = R.member_ID
WHERE R.num_rented > A.avg_rented;
```

- **iv. Status:** Tested and correct.

- **v. Screenshot:**

The screenshot shows a database query editor with the following SQL query:

```
WITH rental_counts AS (
  SELECT member_ID, COUNT(*) AS num_rented
  FROM Rental_Transaction
  GROUP BY member_ID),avg_rent AS (
  SELECT AVG(num_rented) AS avg_rented FROM rental_counts
)
SELECT CM.member_ID, CM.F_name || ' ' || CM.L_name AS member_name, CM.Email,R.num_rented
FROM rental_counts AS R
JOIN avg_rent AS A
JOIN Community_Member AS CM ON CM.member_ID = R.member_ID
WHERE R.num_rented > A.avg_rented;
```

Below the query editor, there is a table view showing the results of the query. The table has 5 columns: member_id, member_name, email, num_rented, and an empty column. The results are as follows:

member_id	member_name	email	num_rented	
132359	Isabella Jackson	isabella.jackson01@mail.com	3	
156304	Riley Lopez	riley.lopez8@mail.com	2	
180657	Elliot Sanchez	elliot.sanchez@mail.com	2	
180913	Lucas Jones	lucas.jones@mail.com	2	
182868	Avery Thompson	averythompsonze@mail.com	2	
187949	James Martinez	james.martinez@mail.com	4	

Query L: Equipment Rental Frequency

- **i. Description:** Lists all equipment with their manufacturer and total times rented, sorted from most rented to least rented.

- **ii. Relational Algebra:**

$$E.equipment_ID, E.Manufacturer \bowtie_{COUNT(RT.equipment_ID) \rightarrow total_times_rented} (Equipment \bowtie_L Rental_Transaction)$$

- **iii. SQL Query:**

```
SELECT E.equipment_ID, E.Manufacturer, COUNT(RT.equipment_ID) AS total_times_rented
FROM Equipment AS E
LEFT JOIN Rental_Transaction AS RT ON E.equipment_ID = RT.equipment_ID
GROUP BY E.equipment_ID, E.Manufacturer
ORDER BY total_times_rented DESC;
```

- **iv. Status:** Tested and correct.

- **v. Screenshot:**

```

1 SELECT E.equipment_ID, E.Manufacturer, COUNT(RT.equipment_ID) AS total_times_rented
2 FROM Equipment AS E
3 LEFT JOIN Rental_Transaction AS RT ON E.equipment_ID = RT.equipment_ID
4 GROUP BY E.equipment_ID, E.Manufacturer
5 ORDER BY total_times_rented DESC;

```

equipment_id	manufacturer	total_times_rented
5080	Fluke	7
5010	Ridgid	5
5007	Ridgid	4
5013	DeWalt	4
5014	Makita	4
5017	Milwaukee	4

Query M: Drone Miles Flown

- i. **Description:** Lists drones with their manufacturer and total miles flown for deliveries, sorted from highest to lowest mileage.
- ii. **Relational Algebra:**

$$D.drone_ID, D.manufacturer \mathrel{\$} SUM(DD.miles_flown) \rightarrow total_miles_flown (Drone \bowtie_{D.drone_ID=DD.drone_ID} Drone_Performs_Delivery)$$

- iii. **SQL Query:**

```

SELECT D.drone_ID, D.manufacturer, SUM(DD.miles_flown) AS total_miles_flown
FROM Drone AS D
JOIN Drone_Performs_Delivery AS DD ON D.drone_ID = DD.drone_ID
GROUP BY D.drone_ID, D.manufacturer
ORDER BY total_miles_flown DESC;

```

- iv. **Status:** Tested and correct.
- v. **Screenshot:**

```

98 SELECT D.drone_ID, D.manufacturer, SUM(DD.miles_flown) AS total_miles_flown
99 FROM Drone AS D
100 JOIN Drone_Performs_Delivery AS DD ON D.drone_ID = DD.drone_ID
101 GROUP BY D.drone_ID, D.manufacturer
102 ORDER BY total_miles_flown DESC;

```

drone_id	manufacturer	total_miles_flown
1038	DJI	290.81
1009	Holy Stone	236.23
1002	Holy Stone	227.73
1001	Autel Robotics	203.08
1005	DJI	196.4
1023	Hubsan	193.59

Query N: Most Popular Manufacturer

- i. **Description:** Identifies the manufacturer with the most rented equipment items.
- ii. **Relational Algebra:**

$$R_{man_counts} \leftarrow Manufacturer \mathrel{\$} COUNT(equipment_ID) \rightarrow rented_count (Equipment \bowtie Rental_Transaction)$$

$$V_{max} \leftarrow \mathrel{\$} MAX(rented_count) (R_{man_counts})$$

$$\pi_{Manufacturer} (R_{man_counts} \bowtie_{rented_count=V_{max}} V_{max})$$

- **iii. SQL Query:**

```
SELECT Manufacturer
FROM (
  SELECT E.Manufacturer, COUNT(RT.equipment_ID) AS rented_count
  FROM Equipment AS E
  JOIN Rental_Transaction AS RT ON E.equipment_ID = RT.equipment_ID
  GROUP BY E.Manufacturer
  ORDER BY rented_count DESC
)
LIMIT 1;
```

- **iv. Status:** Tested and correct.

- **v. Screenshot:**



Query O: Most Used Items by Miles

- **i. Description:** Finds the most used drone models based on total miles flown and number of deliveries, ordered by highest mileage.

- **ii. Relational Algebra:**

$$D.drone_ID, D.manufacturer \bowtie \Sigma_{SUM(DD.miles_flown) \rightarrow total_usage_miles, COUNT(DD.rental_id) \rightarrow times_used} (Drone \bowtie_{D.drone_ID=DD.d}$$

- **iii. SQL Query:**

```
SELECT D.drone_ID AS equipment_ID,
  D.manufacturer,
  SUM(DD.miles_flown) AS total_usage_miles,
  COUNT(DD.rental_id) AS times_used
FROM Drone AS D
JOIN Drone_Performs_Delivery AS DD ON D.drone_ID = DD.drone_ID
GROUP BY D.drone_ID, D.manufacturer
ORDER BY total_usage_miles DESC;
```

- **iv. Status:** Tested and correct.

- **v. Screenshot:**

```

19
20 SELECT D.drone_ID AS equipment_ID,
21        D.manufacturer,
22        SUM(DD.miles_flown) AS total_usage_miles,
23        COUNT(DD.rental_id) AS times_used
24 FROM Drone AS D
25 JOIN Drone_Performs_Delivery AS DD ON D.drone_ID = DD.drone_ID
26 GROUP BY D.drone_ID, D.manufacturer
27 ORDER BY total_usage_miles DESC;

```

equipment_ID	manufacturer	total_usage_miles	times_used
1038	DJI	290.81	9
1009	Holy Stone	236.23	7
1002	Holy Stone	227.73	10
1001	Autel Robotics	203.08	6
1005	DJI	196.4	8
1023	Hubsan	193.59	6

Query P: Members Who Rented Most Demanded Equipment

- **i. Description:** Lists member ID, name, and phone number for members who have rented the most frequently rented equipment.
- **ii. Relational Algebra:**

$$R_{equip_counts} \leftarrow equipment_ID \mathrel{\$} COUNT(*) \rightarrow rental_count (Rental_Transaction)$$

$$R_{most_demand} \leftarrow \pi_{equipment_ID} (R_{equip_counts} \bowtie_{rental_count} \mathrel{\$} MAX(rental_count) (R_{equip_counts}))$$

$$\pi_{CM.member_ID, member_name, CM.Phone} (Rental_Transaction \bowtie R_{most_demand} \bowtie Community_Member)$$

- **iii. SQL Query:**

```

WITH equip_counts AS (
    SELECT equipment_ID, COUNT(*) AS rental_count
    FROM Rental_Transaction
    GROUP BY equipment_ID
),
most_demand AS (
    SELECT equipment_ID
    FROM equip_counts
    WHERE rental_count = (SELECT MAX(rental_count) FROM equip_counts)
)
SELECT DISTINCT CM.member_ID, CM.F_name || ' ' || CM.L_name AS member_name, CM.Phone
FROM Rental_Transaction AS RT
JOIN most_demand AS MD ON RT.equipment_ID = MD.equipment_ID
JOIN Community_Member AS CM ON RT.member_ID = CM.member_ID;

```

- **iv. Status:** Tested and correct.
- **v. Screenshot:**

```

29 WITH equip_counts AS (
30     SELECT equipment_ID, COUNT(*) AS rental_count
31     FROM Rental_Transaction
32     GROUP BY equipment_ID
33 ),
34 most_demand AS (
35     SELECT equipment_ID
36     FROM equip_counts
37     WHERE rental_count = (SELECT MAX(rental_count) FROM equip_counts)
38 )
39 SELECT DISTINCT CM.member_ID, CM.F_name || ' ' || CM.L_name AS member_name, CM.Phone
40 FROM Rental_Transaction AS RT
41 JOIN most_demand AS MD ON RT.equipment_ID = MD.equipment_ID
42 JOIN Community_Member AS CM ON RT.member_ID = CM.member_ID;

```

Grid view Form view

Total rows loaded: 7

member_id	member_name	phone
207581	Peyton Hernandez	817-934-6981
284229	Jamie Wilson	606-959-0693
341192	Emma Hernandez	870-901-5530
539122	Jamie Hernandez	826-682-1020
662093	Amelia Martin	639-201-5780
781463	Riley Lewis	687-257-0154

Query Q: Manufacturers of Equipment Rented by Above-Average Members

- i. **Description:** Lists distinct manufacturers whose equipment was rented by members who have rented more items than the average member.

$$R_{counts} \leftarrow member_id \mathrel{\$} COUNT(*) \rightarrow num_rented (Rental_Transaction)$$

$$R_{above_avg} \leftarrow \pi_{member_id} (R_{counts} \bowtie_{num_rented > avg} \mathrel{\$} AVG(num_rented) \rightarrow avg (R_{counts}))$$

$$\pi_{DISTINCT E.manufacturer} (Equipment \bowtie Rental_Transaction \bowtie R_{above_avg})$$

- iii. **SQL Query:**

```

WITH rental_counts AS (
    SELECT member_id, COUNT(*) AS num_rented
    FROM Rental_Transaction
    GROUP BY member_id
),
above_avg AS (
    SELECT member_id
    FROM rental_counts
    WHERE num_rented > (SELECT AVG(num_rented) FROM rental_counts)
)
SELECT DISTINCT E.manufacturer
FROM Rental_Transaction AS RT
JOIN above_avg AS A ON A.member_id = RT.member_id
JOIN Equipment AS E ON E.equipment_id = RT.equipment_id;

```

- iv. **Status:** Tested and correct.
- v. **Screenshot:**

```

14 WITH rental_counts AS (
15     SELECT member_id, COUNT(*) AS num_rented
16     FROM Rental_Transaction
17     GROUP BY member_id
18 ),
19 above_avg AS (
20     SELECT member_id
21     FROM rental_counts
22     WHERE num_rented > (SELECT AVG(num_rented) FROM rental_counts)
23 )
24 SELECT DISTINCT E.manufacturer
25 FROM Rental_Transaction AS RT
26 JOIN above_avg AS A ON A.member_id = RT.member_id
27 JOIN Equipment AS E ON E.equipment_id = RT.equipment_id;

```

Grid view Form view

Total rows loaded: 8

manufacturer
Milwaukee
Ridgid
Hikmicro
Makita
DeWalt
Ryobi

2.3 INSERT Syntax

When populating the database, you must respect Referential Integrity. Data must be inserted in the following order:

1. **Independent Entities:** Warehouse, Purchase_Order.
2. **Dependent Entities:** Employee (needs Warehouse), Community_Member (needs Warehouse), Drone/Equipment (needs Warehouse and Purchase_Order).
3. **Transactions:** Rental_Transaction (needs Member and Equipment).

Example Insert Statements

Adding a New Warehouse:

```

INSERT INTO Warehouse (wrhs_address, phone, manager_ssn, equipment_cap, drone_cap)
VALUES ('123 Tech Blvd, Columbus, OH 43210', '555-0199', NULL, 500, 50);
-- Note: Manager SSN is NULL initially until an Employee is created.

```

Adding a New Community Member:

```

INSERT INTO Community_Member (member_id, wrhs_address, f_name, l_name, mbr_address, phone, email, start_date,
mbr_status, wrhs_distance)
VALUES (999001, '123 Tech Blvd, Columbus, OH 43210', 'Jane', 'Doe', '456 High St', '555-1234', 'jane.doe@email.com',
DATE('now'), 'Active', 5);

```

Adding a New Rental Transaction:

```

INSERT INTO Rental_Transaction (rental_id, checkout_date, equipment_id, member_id, due_date, rental_fee)
VALUES (200001, DATETIME('now'), 5001, 999001, DATE('now', '+3 days'), 45.00);

```

2.4 DELETE Syntax

Deletions must occur in **reverse order** of insertion to avoid foreign key constraint violations. You cannot delete a Member if they have active Rentals.

Example Delete Statements

Deleting a Rental Transaction (Child):


```
-- Must happen before deleting the Member or Equipment involved  
DELETE FROM Rental_Transaction WHERE rental_id = 200001;
```

Deleting a Drone (Parent/Child Dependency):

```
-- First, remove flight logs associated with the drone  
DELETE FROM Drone_Performs_Delivery WHERE drone_id = 1001;  
DELETE FROM Drone_Performs_Return WHERE drone_id = 1001;  
-- Then, delete the drone record  
DELETE FROM Drone WHERE drone_id = 1001;
```

Deleting a Community Member:

```
-- Ensure all their rentals and reviews are deleted first  
DELETE FROM Rating_And_Review WHERE member_id = 999001;  
DELETE FROM Rental_Transaction WHERE member_id = 999001;  
-- Finally, delete the member  
DELETE FROM Community_Member WHERE member_id = 999001;
```

2.5 Screenshots for Program

Rental Menu:

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

Select an option:
(1): Create a new rental transaction.
(2): Register an equipment return.
(3): Schedule a delivery.
(4): Schedule a pickup.
(0): Back to main menu.
> 1
Enter checkout_date: 2025-12-01
Enter rental_id: 100200
Enter equipment_id: 5045
Enter member_id: 511612
Enter due_date: 2026-02-01
Enter return_date: 2025-12-03
Enter rental_fee: 50
1 row(s) inserted into Rental_Transaction.
Added new Rental_Transaction record.
Rental transaction created.
Select an option:
(1): Create a new rental transaction.
(2): Register an equipment return.
(3): Schedule a delivery.
(4): Schedule a pickup.
(0): Back to main menu.
> 
```

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

Select an option:
(1): Create a new rental transaction.
(2): Register an equipment return.
(3): Schedule a delivery.
(4): Schedule a pickup.
(0): Back to main menu.
> 2
Enter rental ID to record a return for (or 'q' to cancel): 11
No matching record found. Please enter an existing ID.
Enter rental ID to record a return for (or 'q' to cancel): 10011
No matching record found. Please enter an existing ID.
Enter rental ID to record a return for (or 'q' to cancel): 100011
Enter return date (YYYY-MM-DD) (or 'q' to cancel): 2025-12-03
Return recorded for rental 100011.
Select an option:
(1): Create a new rental transaction.
(2): Register an equipment return.
(3): Schedule a delivery.
(4): Schedule a pickup.
(0): Back to main menu.
> 
```

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

Return recorded for rental 100011.
Select an option:
  (1): Create a new rental transaction.
  (2): Register an equipment return.
  (3): Schedule a delivery.
  (4): Schedule a pickup.
  (0): Back to main menu.
> 3
Enter rental ID to schedule a delivery for (or 'q' to cancel): 100025
Enter drone ID to assign to this delivery (or 'q' to cancel): 1000
Enter miles flown (DECIMAL 10,2) (or 'q' to cancel): 27.2
1 row(s) inserted into Drone_Performs_Delivery.
Drone 1000 assigned.
Delivery scheduled for rental 100025.
Select an option:
  (1): Create a new rental transaction.
  (2): Register an equipment return.
  (3): Schedule a delivery.
  (4): Schedule a pickup.
  (0): Back to main menu.
> 
```

```
Select an option:
  (1): Create a new rental transaction.
  (2): Register an equipment return.
  (3): Schedule a delivery.
  (4): Schedule a pickup.
  (0): Back to main menu.
> 4
Enter rental ID to schedule a pickup for (or 'q' to cancel): 100200
Enter drone ID to assign to this pickup (or 'q' to cancel): 1000
1 row(s) inserted into Drone_Performs_Return.
Drone 1000 assigned.
Pickup scheduled for rental 100200.
Select an option:
  (1): Create a new rental transaction.
  (2): Register an equipment return.
  (3): Schedule a delivery.
  (4): Schedule a pickup.
  (0): Back to main menu.
> 
```

New entity option (same generic class handles all new entity options):

```

Welcome to Drone Delivery DBMS!
Select an option:
(1): Manage Rentals
(2): Create a new entity.
(3): Edit or delete an entity.
(4): Search for an entity.
(5): List all entities and records.
(6): Generate Reports.
(0): Exit
> 2
Select an entity type:
1 - Drone
2 - Equipment
3 - Rental Transaction
4 - Community Member
5 - Drone Performs Delivery
6 - Drone Performs Return
7 - Drone Undergoes Maintenance
8 - Employee
9 - Equipment Undergoes Maintenance
10 - Maintenance Record
11 - Purchase Order
12 - Rating and Review
13 - Warehouse
14 - Warehouse Houses Drone
0 - Back
1
Enter manufacturer: Parrot
Enter drone_id: 1040
Enter year: 2025
Enter load_capacity: 15
Enter warranty_expir: 2030-01-01
Enter wrhs_address: 9774 Spring St, Columbus, OH 43002 #2
Enter order_num: 12345
1 row(s) inserted into Drone.
Added new Drone record.
Welcome to Drone Delivery DBMS!
Select an option:
(1): Manage Rentals
(2): Create a new entity.
(3): Edit or delete an entity.

```

Edit record path (same class handles edits for all entity types):

Welcome to Drone Delivery DBMS!

Select an option:

- (1): Manage Rentals
- (2): Create a new entity.
- (3): Edit or delete an entity.
- (4): Search for an entity.
- (5): List all entities and records.
- (6): Generate Reports.
- (0): Exit

> 3

Select an entity type:

- 1 - Drone
- 2 - Equipment
- 3 - Rental Transaction
- 4 - Community Member
- 5 - Drone Performs Delivery
- 6 - Drone Performs Return
- 7 - Drone Undergoes Maintenance
- 8 - Employee
- 9 - Equipment Undergoes Maintenance
- 10 - Maintenance Record
- 11 - Purchase Order
- 12 - Rating and Review
- 13 - Warehouse
- 14 - Warehouse Houses Drone
- 0 - Back

1

```
[0] manufacturer=Autel Robotics, drone_id=1000, year=2024, load_capacity=11.53, warranty_expir=2028-02-20, wrhs_address=1684 Main St, Gahanna, OH 43001 #5, order_num=45554
[1] manufacturer=Autel Robotics, drone_id=1001, year=2026, load_capacity=17.74, warranty_expir=2028-01-12, wrhs_address=9774 Spring St, Columbus, OH 43002 #2, order_num=24538
[2] manufacturer=Holy Stone, drone_id=1002, year=2025, load_capacity=19.27, warranty_expir=2029-03-27, wrhs_address=3357 Indianola Ave, Westerville, OH 43210 #3, order_num=12187
[3] manufacturer=Autel Robotics, drone_id=1003, year=2026, load_capacity=4.07, warranty_expir=2030-02-23, wrhs_address=7024 Main St, Reynoldsburg, OH 43004 #4, order_num=82465
[4] manufacturer=DJI, drone_id=1004, year=2026, load_capacity=23.24, warranty_expir=2029-04-03, wrhs_address=1391 3rd St, Reynoldsburg, OH 43215 #6, order_num=70807
[5] manufacturer=DJI, drone_id=1005, year=2025, load_capacity=20.75, warranty_expir=2028-02-23, wrhs_address=3357 Indianola Ave, Westerville, OH 43210 #3, order_num=78084
[6] manufacturer=Skydio, drone_id=1006, year=2025, load_capacity=14.31, warranty_expir=2027-04-17, wrhs_address=1684 Main St, Gahanna, OH 43001 #5, order_num=65163
```

```
, Reynoldsburg, OH 43004 #1, order_num=66948
[34] manufacturer=Parrot, drone_id=1034, year=2024, load_capacity=18.34, warranty_expir=2026-03-24, wrhs_address=3833 Olentangy
River Rd, Reynoldsburg, OH 43004 #7, order_num=51288
[35] manufacturer=Autel Robotics, drone_id=1035, year=2025, load_capacity=4.87, warranty_expir=2029-03-01, wrhs_address=7024 Mai
n St, Reynoldsburg, OH 43004 #4, order_num=50306
[36] manufacturer=Yuneec, drone_id=1036, year=2026, load_capacity=17.71, warranty_expir=2028-02-29, wrhs_address=3357 Indianola
Ave, Westerville, OH 43210 #3, order_num=77862
[37] manufacturer=Yuneec, drone_id=1037, year=2025, load_capacity=10.14, warranty_expir=2027-02-06, wrhs_address=3833 Olentangy
River Rd, Reynoldsburg, OH 43004 #7, order_num=55406
[38] manufacturer=DJI, drone_id=1038, year=2026, load_capacity=9.68, warranty_expir=2029-04-25, wrhs_address=1391 3rd St, Reynol
dsburg, OH 43215 #6, order_num=50654
[39] manufacturer=Hubsan, drone_id=1039, year=2024, load_capacity=3.83, warranty_expir=2027-04-25, wrhs_address=1924 High St, Re
ynoldsburg, OH 43004 #1, order_num=22475
[40] manufacturer=Parrot, drone_id=1040, year=2025, load_capacity=15, warranty_expir=2030-01-01, wrhs_address=9774 Spring St, Co
lumbus, OH 43002 #2, order_num=12345
Enter record ID (0 to 40), or 'c' to cancel: 40
Select an action:
  (1): Edit
  (2): Delete
> 1
Available fields:
  manufacturer
  drone_id
  year
  load_capacity
  warranty_expir
  wrhs_address
  order_num
Enter field name to edit (exact): year
New value for 'year': 2024
Record updated.
Welcome to Drone Delivery DBMS!
Select an option:
  (1): Manage Rentals
  (2): Create a new entity.
  (3): Edit or delete an entity.
  (4): Search for an entity.
  (5): List all entities and records.
  (6): Generate Reports.
  (0): Exit
>
```

Delete option:

```
[38] manufacturer=DJI, drone_id=1038, year=2026, load_capacity=9.68, warranty_expir=2029-04-25, wrhs_address=1391 3rd St, Reynol
dsburg, OH 43215 #6, order_num=50654
[39] manufacturer=Hubsan, drone_id=1039, year=2024, load_capacity=3.83, warranty_expir=2027-04-25, wrhs_address=1924 High St, Re
ynoldsburg, OH 43004 #1, order_num=22475
[40] manufacturer=Parrot, drone_id=1040, year=2024, load_capacity=15, warranty_expir=2030-01-01, wrhs_address=9774 Spring St, Co
lumbus, OH 43002 #2, order_num=12345
Enter record ID (0 to 40), or 'c' to cancel: 40
Select an action:
  (1): Edit
  (2): Delete
> 2
Record deleted.
Welcome to Drone Delivery DBMS!
Select an option:
  (1): Manage Rentals
  (2): Create a new entity.
  (3): Edit or delete an entity.
  (4): Search for an entity.
  (5): List all entities and records.
  (6): Generate Reports.
  (0): Exit
>
```

Search by substring for a specific entity:


```

Welcome to Drone Delivery DBMS!
Select an option:
  (1): Manage Rentals
  (2): Create a new entity.
  (3): Edit or delete an entity.
  (4): Search for an entity.
  (5): List all entities and records.
  (6): Generate Reports.
  (0): Exit
> 4
Search a specific entity? (y/n): y
Select an entity type:
  1 - Drone
  2 - Equipment
  3 - Rental Transaction
  4 - Community Member
  5 - Drone Performs Delivery
  6 - Drone Performs Return
  7 - Drone Undergoes Maintenance
  8 - Employee
  9 - Equipment Undergoes Maintenance
 10 - Maintenance Record
 11 - Purchase Order
 12 - Rating and Review
 13 - Warehouse
 14 - Warehouse Houses Drone
  0 - Back
8
Enter substring to search for: Garcia
Matches:
[Employee]
f_name, m_name, l_name, ssn, wrhs_address, role, phone_no, email, hire_date, pay_rate, hrs_worked
Mason, null, Garcia, 371777932, 7024 Main St, Reynoldsburg, OH 43004 #4, Coordinator, 973-947-3295, mason.garcial9@compan
y.org, 2025-04-15, 28.08, 45
Welcome to Drone Delivery DBMS!

```

Search by substring across all entities:

```

Welcome to Drone Delivery DBMS!
Select an option:
  (1): Manage Rentals
  (2): Create a new entity.
  (3): Edit or delete an entity.
  (4): Search for an entity.
  (5): List all entities and records.
  (6): Generate Reports.
  (0): Exit
> 4
Search a specific entity? (y/n): n
Enter substring to search for: Parrot
Matches:
[Drone]
manufacturer, drone_id, year, load_capacity, warranty_expir, wrhs_address, order_num
Parrot, 1010, 2026, 12.2, 2028-02-29, 1684 Main St, Gahanna, OH 43001 #5, 10167
Parrot, 1019, 2024, 17.38, 2027-01-16, 9774 Spring St, Columbus, OH 43002 #2, 44735
Parrot, 1024, 2025, 12.31, 2029-04-12, 6077 Neil Ave, Gahanna, OH 43004 #8, 93789
Parrot, 1034, 2024, 18.34, 2026-03-24, 3833 Olentangy River Rd, Reynoldsburg, OH 43004 #7, 51288
Parrot, 1024, 2025, 12.31, 2029-04-12, 6077 Neil Ave, Gahanna, OH 43004 #8, 93789
Parrot, 1024, 2025, 12.31, 2029-04-12, 6077 Neil Ave, Gahanna, OH 43004 #8, 93789
Parrot, 1034, 2024, 18.34, 2026-03-24, 3833 Olentangy River Rd, Reynoldsburg, OH 43004 #7, 51288
Welcome to Drone Delivery DBMS!
Select an option:
  (1): Manage Rentals
  (2): Create a new entity.
  (3): Edit or delete an entity.
  (4): Search for an entity.
  (5): List all entities and records.
  (6): Generate Reports.
  (0): Exit
> █

```

List main option:

```

Welcome to Drone Delivery DBMS!
Select an option:
  (1): Manage Rentals
  (2): Create a new entity.
  (3): Edit or delete an entity.
  (4): Search for an entity.
  (5): List all entities and records.
  (6): Generate Reports.
  (0): Exit
> 5
Select an entity type:
  1 - Drone
  2 - Equipment
  3 - Rental Transaction
  4 - Community Member
  5 - Drone Performs Delivery
  6 - Drone Performs Return
  7 - Drone Undergoes Maintenance
  8 - Employee
  9 - Equipment Undergoes Maintenance
 10 - Maintenance Record
 11 - Purchase Order
 12 - Rating and Review
 13 - Warehouse
 14 - Warehouse Houses Drone
 0 - Back
13
Listing all rows from Warehouse:
phone, wrhs_address, manager_ssn, equipment_cap, drone_cap
342-954-1679, 1924 High St, Reynoldsburg, OH 43004 #1, 693791998, 4967, 21
423-438-8279, 9774 Spring St, Columbus, OH 43002 #2, 755984598, 717, 81
803-484-0106, 3357 Indianola Ave, Westerville, OH 43210 #3, 494834114, 1807, 99
981-544-1674, 7024 Main St, Reynoldsburg, OH 43004 #4, 693791998, 1259, 58
947-670-8785, 1684 Main St, Gahanna, OH 43001 #5, 355198260, 1522, 58
396-921-1139, 1391 3rd St, Reynoldsburg, OH 43215 #6, 370749004, 875, 94
303-589-4554, 3833 Olentangy River Rd, Reynoldsburg, OH 43004 #7, 494834114, 4214, 91
886-473-1169, 6077 Neil Ave, Gahanna, OH 43004 #8, 446124770, 1901, 78
Welcome to Drone Delivery DBMS!
Select an option:
  (1): Manage Rentals
  (2): Create a new entity.
  (3): Edit or delete an entity.
  (4): Search for an entity.
  (5): List all entities and records.
  (6): Generate Reports.
  (0): Exit
> █

```

Report 1:


```

Welcome to Drone Delivery DBMS!
Select an option:
(1): Manage Rentals
(2): Create a new entity.
(3): Edit or delete an entity.
(4): Search for an entity.
(5): List all entities and records.
(6): Generate Reports.
(0): Exit
> 6
Select a report:
(1): Total Rentals by a Member
(2): Most Popular Item
(3): Most Frequent Equipment Manufacturer
(4): Most Used Drone
(5): Member with Most Items Rented
(6): Equipment by Type Released Before a Given Year
(0): Back to main menu
> 1

=== Report 1: Total Rentals by a Member ===
Enter member ID to search (or 'q' to cancel): 805000

Results:
member_id, f_name, l_name, Rentals
805000, Jamie, White, 3
Select a report:
(1): Total Rentals by a Member
(2): Most Popular Item
(3): Most Frequent Equipment Manufacturer
(4): Most Used Drone
(5): Member with Most Items Rented
(6): Equipment by Type Released Before a Given Year
(0): Back to main menu
>

```

Report 2:

```

Select a report:
(1): Total Rentals by a Member
(2): Most Popular Items
(3): Most Frequent Equipment Manufacturer
(4): Most Used Drone
(5): Member with Most Items Rented
(6): Equipment by Type Released Before a Given Year
(0): Back to main menu
> 2

=== Report 2: Most Frequently Rented Equipment ===

Results:
equipment_id, manufacturer, total_times_rented
5080, Fluke, 7
5010, Ridgid, 5
5007, Ridgid, 4
5013, DeWalt, 4
5014, Makita, 4
5017, Milwaukee, 4
5030, Makita, 4
5036, Bosch, 4
5038, Hikmicro, 4
5042, Fluke, 4
5045, Ryobi, 4
5068, Ryobi, 4
5082, Hikmicro, 4

```

```

5032, DeWalt, 0
5040, DeWalt, 0
5051, Makita, 0
5055, Hikmicro, 0
5056, Makita, 0
5064, Makita, 0
5070, DeWalt, 0
5071, Hikmicro, 0
5077, DeWalt, 0
5087, Makita, 0
5089, DeWalt, 0
Select a report:
(1): Total Rentals by a Member
(2): Most Popular Items
(3): Most Frequent Equipment Manufacturer
(4): Most Used Drone
(5): Member with Most Items Rented
(6): Equipment by Type Released Before a Given Year
(0): Back to main menu
> 

```

Report 3:

```

Select a report:
(1): Total Rentals by a Member
(2): Most Popular Items
(3): Most Frequently Used Equipment Manufacturer
(4): Most Used Drone
(5): Member with Most Items Rented
(6): Equipment by Type Released Before a Given Year
(0): Back to main menu
> 3

=== Report 3: Most Frequent Equipment Manufacturer ===

Results:
manufacturer
Milwaukee
Select a report:
(1): Total Rentals by a Member
(2): Most Popular Items
(3): Most Frequently Used Equipment Manufacturer
(4): Most Used Drone
(5): Member with Most Items Rented
(6): Equipment by Type Released Before a Given Year
(0): Back to main menu
> 

```

Report 4:

```

Select a report:
(1): Total Rentals by a Member
(2): Most Popular Items
(3): Most Frequently Used Equipment Manufacturer
(4): Most Used Drone
(5): Member with Most Items Rented
(6): Equipment by Type Released Before a Given Year
(0): Back to main menu
> 4

=== Report 4: Drone Miles Flown ===

Results:
drone_id, manufacturer, total_miles_flown
1038, DJI, 290.81
1009, Holy Stone, 236.23
1002, Holy Stone, 227.73
1001, Autel Robotics, 203.08
1005, DJI, 196.4
1023, Hubsan, 193.59
1014, Hubsan, 193.48

```

```

1032, Hubsan, 49.66
1013, Holy Stone, 47.41
1036, Yuneec, 40.83
1000, Autel Robotics, 27.2
1022, Yuneec, 26.62
Select a report:
(1): Total Rentals by a Member
(2): Most Popular Items
(3): Most Frequently Used Equipment Manufacturer
(4): Most Used Drone
(5): Member with Most Items Rented
(6): Equipment by Type Released Before a Given Year
(0): Back to main menu
>

```

Report 5:

```

Select a report:
(1): Total Rentals by a Member
(2): Most Popular Items
(3): Most Frequently Used Equipment Manufacturer
(4): Most Used Drone
(5): Member with Most Items Rented
(6): Equipment by Type Released Before a Given Year
(0): Back to main menu
> 5

=== Report 5: Member with Most Items Rented ===

Results:
member_id, f_name, l_name, Rentals
800005, Jamie, Anderson, 6
Select a report:
(1): Total Rentals by a Member
(2): Most Popular Items
(3): Most Frequently Used Equipment Manufacturer
(4): Most Used Drone
(5): Member with Most Items Rented
(6): Equipment by Type Released Before a Given Year
(0): Back to main menu
>

```

Report 6:

```

Select a report:
(1): Total Rentals by a Member
(2): Most Popular Items
(3): Most Frequently Used Equipment Manufacturer
(4): Most Used Drone
(5): Member with Most Items Rented
(6): Equipment by Type Released Before a Given Year
(0): Back to main menu
> 6

=== Report 6: Equipment by Type Released Before a Given Year ===
Enter equipment type (or 'q' to cancel): LED work light
Enter cutoff year (e.g., 2020), or 'q' to cancel: 2025

Results:
equipment_id, type, manufacturer, year
5095, LED work light, Makita, 2024
Select a report:
(1): Total Rentals by a Member
(2): Most Popular Items
(3): Most Frequently Used Equipment Manufacturer
(4): Most Used Drone
(5): Member with Most Items Rented
(6): Equipment by Type Released Before a Given Year
(0): Back to main menu
>

```

Exit:

```
Welcome to Drone Delivery DBMS!
Select an option:
(1): Manage Rentals
(2): Create a new entity.
(3): Edit or delete an entity.
(4): Search for an entity.
(5): List all records by entity.
(6): Generate Reports.
(0): Exit
> 0
Goodbye.
Database connection closed.
PS C:\Indp Class Files\CSE3241\DroneDeliveryDBMS>
```

3 Avoiding SQL Injection

In the original version of the project, all "database" operations were performed against in-memory `Map` and `List` objects. When migrating to a real SQLite backend, I implemented several best practices to avoid SQL injection and sanitize input:

- All dynamic values are passed via `PreparedStatement` parameters and are never concatenated directly into SQL.
- Table and column names are not taken from raw user input; instead, they are whitelisted via an `EntityDefinition` enum.
- For updates, the field name to edit is also checked against this whitelist before being used in SQL.
- User input is validated and sanitized (for example, date formats and numeric fields) before being sent to the database.

Example: `createNew` method

The `createNew` method handles insertion for all tables:

```
/* Insert the row into the database using a prepared statement. */
try {
    SQL.insertRow(entityName, attributes, values);
    System.out.println("Added new " + entityName + " record.");
} catch (SQLException e) {
    System.out.println("Error inserting new " + entityName + ": " + e.getMessage());
}
```

Here, all user data comes through the `values[]` array and is bound as parameters. This treats all user input as data (strings), not as executable SQL.

Example: `EntityDefinition` enum

Table and column names are whitelisted because all table names and attributes come from the `EntityDefinition` enum. For example:

```
public enum EntityDefinition {
    DRONE(
        1, "Drone", "Drone",
        new String[] { "manufacturer", "drone_id", "year", ... }
    ),
    EQUIPMENT(
        2, "Equipment", "Equipment",
        new String[] { "type", "year", "order_num", ... }
    ),
    ...
}
```

The enum is used to map a user-provided type index to a valid table name:

```
public static String getEntityNameFromIndex(String type) {
    try {
        int idx = Integer.parseInt(type);
        EntityDefinition def = EntityDefinition.fromIndex(idx);
        return (def != null) ? def.getTableName() : null;
    } catch (NumberFormatException e) {
        return null;
    }
}
```

The same practice is applied to the way updates are handled.

Finally, the program also validates input types and formats (for example, numeric years and `YYYY-MM-DD` dates) before passing data to the database.

4 Submitted Files Specifications

- **4.1 Database File(s)**

- `drone_based_equitment_rental_database.db`

SQLite binary database file.

- `DroneDeliveryDBS.sql`

SQL script used to populate the database.

- **4.2 SQL Scripts (to Recreate Database)**

- `AllQueries.txt`

Contains all SQL queries used in the project (including sample and advanced queries).

- **4.3 Program (Software Code)**

- ``DroneDeliveryDBMS.zip``

Java program implementing the user interface and connecting to the SQLite database.

- **4.4 Program Deliverables & How to Run in VS Code**

- **Importing the Project Into VS Code**

1. **Unzip the project folder**

- Unzip the entire project archive so that the source files and directory structure are preserved.
- Make sure you know where the unzipped folder is located.

2. **Open the project folder in VS Code**

- Launch VS Code.
- Go to the **File** menu → **Open Folder...**
- Select the unzipped project folder (the folder that contains `/src`, `/options`, `/utilities`, etc.).

- Click **Open**.

3. **Allow VS Code to build the project**

- When prompted with messages such as "Import Java project?" or "Build workspace?", choose **Yes** for all prompts.